Module 4 — Databases, SQL, NoSQL, Transactions and CRUD

Vistula University · IT Fundamentals · Theory handout

Vistula University · IT Fundamentals

Learning Objectives

After this module, you will be able to:

- Explain what a database is and why a web application needs one.
- Tell relational and non-relational databases apart.
- Write simple SQL: SELECT, INSERT, UPDATE, DELETE.
- Explain tables, rows, columns, primary keys, and foreign keys.
- Describe a one-to-many relationship between two tables.
- Explain what a transaction is and what the four ACID letters mean.
- Describe what Supabase provides as one service.
- Explain in simple words what Row Level Security (RLS) is for.

---

1. What Is a Database?

A **database** is a place where a program keeps its data in an organized way, so the data can be found, changed, and saved again quickly and safely.

Think about our **Student Task Manager**. It has a table called `tasks`. Each task has a title, a "done" flag, and a date.

In Module 3, we saved tasks in **localStorage** — a small storage area inside the browser. It is good for learning, but it is **not enough** for a real application:

- **Not shared.** Your tasks live only in your browser. You cannot see them on another device, and two people can never see the same list.
- **Not safe.** Anyone using the same browser can read or change localStorage. There are no user accounts and no permissions.
- **Not central.** Data that must be the same for many users needs one central place. A database is that place.

A database runs on a **server** (a computer that is always on). The application talks to the database, and the database answers.

---

2. Relational vs Non-Relational Databases

There are two big families of databases.

A **relational database** stores data in **tables**. A table looks like a spreadsheet: data goes into rows and columns, and every row has the same shape. Tables can point to each other. Examples: PostgreSQL, MySQL, SQLite.

A **non-relational database** (often called **NoSQL**) stores data in other shapes. The most popular shape is a **document**: one block of data, usually in JSON format, like an object in JavaScript. There is no fixed table shape — two documents can have different fields. MongoDB is a famous example.

SQL vs NoSQL

SQL means *Structured Query Language*. It is the language we use to talk to relational databases; the name "SQL" is also used for the whole family of relational databases. **NoSQL** databases do not use classic SQL tables.

There is no single "best" choice. Banks and shops often choose relational databases, because they need strict rules and consistency. Applications with huge amounts of simple, fast-changing data often choose NoSQL.

Popular Examples

- **PostgreSQL** — a powerful, free relational database. It is used by Supabase (see below).
- **MySQL** — another very popular free relational database. It powers many websites.
- **SQLite** — a tiny relational database that lives inside one file. Perfect for mobile apps and learning.
- **MongoDB** — a document NoSQL database. It stores JSON-like documents.

---

3. Tables, Rows, and Columns

Relational data lives in tables. For our Student Task Manager:

- A **table** is one kind of data. Example: the `tasks` table.
- A **row** is one item. Example: one task, "Buy milk".
- A **column** is one property of the item. Example: `title`, `done`, `created_at`.

The `tasks` table might look like this:

id	user_id	title	done	created_at
1	7	Buy milk	false	2026-10-14 09:00
2	7	Read chapter 4	true	2026-10-14 10:30

Primary Keys and Foreign Keys

A **primary key** is a column that gives every row its own unique number or code. No two rows can share it. In our table, `id` is the primary key. With it, you can always find exactly one row — very important for UPDATE and DELETE.

A **foreign key** is a column that holds the primary key of another table. It "points" to a row there. In `tasks`, the column `user_id` points to a row in a `users` table: "this task belongs to this user".

Relationships Between Tables

When a foreign key links two tables, we call this a **relationship**. Our example is a **one-to-many** relationship: one **user** can have **many tasks**, but each task belongs to only **one** user.

```
users (one)          tasks (many)
+----+-------+       +----+---------+----------+
| id | name  |       | id | user_id | title    |
+----+-------+       +----+---------+----------+
```

```
| 7  | Anna  | <---- | 1  | 7       | Buy milk |
| 8  | Piotr | <---- | 2  | 7       | Read ch4 |
```

Look at the `user_id` values: user 7 (Anna) owns both tasks. That is one-to-many.

---

4. SQL: The Four Basic Commands

You write SQL as short commands. Here are the four most important ones.

SELECT — read data

```
SELECT * FROM tasks;
```

This says: give me all columns (`*`) from the `tasks` table. To get only Anna's tasks that are not done:

```
SELECT * FROM tasks
WHERE user_id = 7 AND done = false;
```

`WHERE` means "only rows that match this condition".

INSERT — add a new row

```
INSERT INTO tasks (user_id, title)
VALUES (7, 'Buy milk');
```

UPDATE — change existing rows

```
UPDATE tasks
SET done = true
WHERE id = 1;
```

Be careful: without `WHERE`, UPDATE changes **every** row in the table!

DELETE — remove rows

```
DELETE FROM tasks
WHERE id = 2;
```

Again, `WHERE` decides which rows go. Without it, you delete everything.

CRUD — the Four Everyday Actions

Every application that stores data does the same four actions. They have a short name: **CRUD**.

Letter	Action	SQL command	In our app
C	Create	`INSERT`	Add a new task
R	Read	`SELECT`	Show the task list
U	Update	`UPDATE`	Mark a task done

Letter	Action	SQL command	In our app
D	Delete	`DELETE`	Remove a task

---

5. Transactions and ACID

A **transaction** is a group of database actions that must succeed or fail **together** — all of them, or none of them.

The classic example is a **bank transfer**. You send 100 zł to a friend. Two things must happen:

1. Money (−100 zł) leaves your account.
2. Money (+100 zł) arrives in your friend's account.

Imagine the power goes off after step 1. Your money disappeared, and your friend got nothing! A transaction prevents this. If **both** steps work, the database saves both. If **anything** fails, it undoes everything — your 100 zł stay in your account.

ACID

Transactions follow four rules. The first letters spell **ACID**.

- **A — Atomicity.** The transaction is one unit. All actions happen, or none happen. No half-done work.
- **C — Consistency.** The database moves from one correct state to another correct state. Rules are never broken — for example, money never appears from nowhere.
- **I — Isolation.** Many transactions can run at the same time without mixing each other up. Two transfers at the same moment do not corrupt each other.
- **D — Durability.** When the database says "saved", the data is really saved. Even a power cut will not lose it.

Why Data Consistency Matters

Consistency means the data is always correct and complete: every task points to a real user, no duplicate primary keys. If an application shows wrong data, users stop trusting it. This is why databases have rules (keys, types, transactions) that protect the data even when the program has a bug.

---

6. Supabase: Your Database in the Cloud

Supabase is an online service that gives you, in one place:

- a real **PostgreSQL** database (the relational database we met above),
- **authentication** — ready-made user accounts: sign up, log in, user ids,
- **permissions** — rules that say who may read or change which data.

You install nothing: you get a project **URL** and a **key**, and your web page talks to Supabase over the internet. In the lab, you will connect the Student Task Manager to it.

Access Permissions and Row Level Security

Access permissions are rules about who may do what: a logged-in student may change their own tasks but not Anna's tasks.

Row Level Security (RLS) is a PostgreSQL feature for fine-grained permissions: security rules that work **row by row**. A rule decides, for every single row, **WHO** can see or change **WHAT**. A policy (a rule) is written as a condition. In words, our rule is:

> "A user may see or change a row in `tasks` only when the row's `user_id` equals the id of the logged-in user."

In SQL, the same rule looks like this:

```
CREATE POLICY "users manage own tasks"
ON tasks FOR ALL
USING (auth.uid() = user_id);
```

Here `auth.uid()` is a Supabase function that returns the id of the logged-in user. So if Anna is logged in, she can only touch rows where `user_id` is Anna's id. Piotr's rows are invisible to her — protected by the database itself, not by our JavaScript code.

Key Terms (Glossary)

Term	Simple meaning (English)	Po polsku
database	An organized place where a program stores its data	baza danych
relational database	A database that stores data in tables connected by keys	baza relacyjna
NoSQL	A non-relational database; stores data as documents or other flexible shapes	baza nierelacyjna (NoSQL)
SQL	Structured Query Language — the language for asking a database for data	język zapytań SQL
table	One set of similar data, like a spreadsheet	tabela
row	One item in a table, for example one task	wiersz
column	One property of the items, for example `title`	kolumna
primary key	A column with a unique value for every row	klucz główny
foreign key	A column that points to the primary key of another table	klucz obcy
relationship	A link between two tables through keys	relacja, powiązanie
transaction	A group of actions that all succeed together or all fail together	transakcja
ACID	The four safety rules for transactions: Atomicity, Consistency, Isolation, Durability	ACID (niepodzielność, spójność, izolacja, trwałość)
CRUD	Create, Read, Update, Delete — the four basic data actions	CRUD (tworzenie, odczyt, aktualizacja, usuwanie)
consistency	The data is always correct and follows all rules	spójność danych
Row Level Security	Database rules that decide row by row who can see or change what	zabezpieczenia na poziomie wierszy
PostgreSQL	A popular, free, open-source relational database	PostgreSQL (popularna baza relacyjna)

Summary

- A database is a safe, central, shared place for data; localStorage is not enough for real applications.
- Relational databases store data in tables; NoSQL databases store flexible documents.
- SQL: SELECT reads, INSERT adds, UPDATE changes, DELETE removes.
- The primary key identifies each row; a foreign key links two tables (one user → many tasks).
- CRUD maps to SQL: Create = INSERT, Read = SELECT, Update = UPDATE, Delete = DELETE.
- A transaction succeeds or fails as a whole; ACID keeps data safe and correct.
- Supabase gives you PostgreSQL + user accounts + permissions as one cloud service.
- Row Level Security protects data row by row: users see and change only their own rows.